are in a venue from Rails to React. I'll talk more about that in Chapter 8, *Talking to the Server*.

With all our components done, we need to connect them into the page so that React is invoked to control the section of the page we want it to draw. That means we want a top-level JavaScript file that we include in our page that will embed the React components into the existing DOM. This is slightly different from many React apps that take over the entire page. In our case, React is, at least for the moment, only managing our grid of seats. Here's our file,

chapter_04/02/app/javascript/components/venue_display.tsx

```
import * as React from "react"
import { createRoot } from "react-dom/client"
import Venue from "./venue"

document.addEventListener("turbo:load", () => {
  if (document.getElementById("react-element")) {
    const container = document.getElementById("react-element")
    const root = createRoot(container)
    root.render(<Venue rows={10} seatsPerRow={10} />)
  }
})
```

It starts by importing React and a method called `createRoot` from ReactDOM, which is the library that allows React to interact with the actual browser DOM. It also imports our `Venue` component.

Then there's a standard DOM event listener, which is waiting for the `turbo:load` event to signify that the DOM has completely loaded for the page. Since we are using Turbo Drive, we won't get an ordinary DOM page load event because the page doesn't technically load; Turbo Drive just inserts the body into the DOM. Turbo Drive provides the `turbo:load` event as an equivalent.

When the event fires, it checks to see if the DOM element we want to connect to exists, which in our case is a DOM element with the ID `react-element`. If so, the code calls the `createRoot` method, with the DOM element as an argument. The `createRoot` method allows React to set up the element as the base of the React app